APPENDIX

A. Java and Smali Block Segmentation

Figures 4 and 5 illustrate the block segmentation of a Java and Smali class, respectively.

Java Method-Level Segmentation Example

Original

```
import google.api;
class A {
    int x;
    void a() {
        y = 1;
    }
    void b() {
        str s = "xyz";
    }
}
```

HeaderBlock

```
import google.api;
class A {
    int x;
}
```

MethodBlock_1

```
void a() {
    y = 1;
}
```

MethodBlock_2

```
void b() {
    str s = "xyz";
}
```

Fig. 4: Method-level segmentation of a Java class into HEADER and METHOD segments.

Smali Method-Level Segmentation Example

Original

```
.class public Lcom/A;
.super Ljava/lang/Object;

.field x:I

.method public a()V
 const/4 v0, 0x1
 return-void
.end method

.method public b()V
 return-void
.end method
```

HeaderBlock

```
.class public Lcom/A;
.super Ljava/lang/Object;
.field x:I
```

MethodBlock_1

```
.method public a()V
 const/4 v0, 0x1
 return-void
.end method
```

MethodBlock_2

```
.method public b()V
 return-void
.end method
```

Fig. 5: Method-level segmentation of a Smali class into HEADER and METHOD segments.

B. Java Pre-Processing and Semantic Filtering

1) Pre-Processing: To ensure that the Java source used for analysis is free from decompiler artifacts, we applied a series of regular-expression filters to remove autogenerated comments, metadata blocks, and diagnostic markers introduced by `jadx`. These elements do not contribute to program semantics and may interfere with feature extraction. Table XV lists the patterns used to sanitize the decompiled Java files.

TABLE XV: Regular expressions used for removing autogenerated Java artifacts

Pattern	Purpose
`(?s)/\*.*?JADX.*?\*/`	Removes multi-line `JADX`-generated header comments.
`//\s *JADX[^\n ]*`	Removes single-line decompiler notes prefixed by `// JADX`.
`//\s *(synthetic|bridge)\b .*`	Filters comments describing synthetic or bridge methods.
`//\s *(Code|Instructions):.*`	Removes placeholder markers inserted by the decompiler.
`/\*\s *compiled from:.*?\*/`	Removes metadata blocks such as "compiled from".

2) Semantic Filtering: Java semantic filtering is implemented using a sequence of rule-based regular-expression transformations. The process removes non-functional content (e.g., comments, decompiler warnings, and autogenerated markers) and normalizes unstable program elements into canonical placeholder tokens while preserving stable API identifiers. The substitution categories are summarized in Table I.

The implementation applies regular-expression rules sequentially to perform comment removal, literal normalization, package anonymization, and identifier substitution. Representative examples are shown below.

Example Regex Patterns

```
# Remove block comments
re.sub(r'/\*.*?\*/', ' ', code, flags=re.DOTALL)

# Remove line comments
re.sub(r'//.*?$', ' ', code, flags=re.MULTILINE)

# Replace string and character literals
re.sub(r'"(?:\\.|[^"\\])*"', 'STR', code)
re.sub(r"'\w'", 'STR', code)

# Replace numeric literals
re.sub(r'\b\d+(\.\d+)?\b', 'NUM', code)

# Package anonymization (non-standard packages)
re.sub(r'package\s+([^\s;]+);', repl_package_import, code)
re.sub(r'import\s+([^\s;]+);', repl_package_import, code)
```

Together, these transformations produce a normalized representation that is less sensitive to identifier renaming and other source-level obfuscation while preserving semantic information used for library matching.

C. Smali Preprocessing and Semantic Filtering

1) Pre-Processing: As with Java, Smali segments may contain redundant decompiler metadata and debug information, which are removed via the same preprocessing step applied separately to each METHOD and HEADER segment.

2) Semantic Filtering: Smali semantic filtering is implemented using rule-based instruction filtering. Each METHOD and HEADER segment is processed independently, retaining only instructions that capture structural and behavioral semantics while removing debug information, redundant directives, annotation blocks, and other non-functional metadata [40]–[42].

TABLE XVI: Smali semantic filtering rules

Pattern	Semantic Purpose
`.method`, `.end method`	Method boundaries
`.super`	Class hierarchy
`.implements`	Interface relationships
`.field`	Field declarations
`invoke-*`	Method invocation behaviour
`const-string`	String-loading operations
`iget`, `iput`	Instance field accesses
`sget`, `sput`	Static field accesses
`L...;`	Class type references

The retained instruction categories preserve method boundaries, class hierarchy, field accesses, method invocations, string-loading operations, and type references, providing a normalized representation that remains stable across compiler optimizations and obfuscation.

TABLE XVIII: Complete hyperparameter sensitivity analysis using 5-fold cross-validation with DS2 Apps (R8-obf-shr-rcs-opt and R8-obf-shr-opt).

α	Fold 1 (Th1, Th2) F1	Fold 2 (Th1, Th2) F1	Fold 3 (Th1, Th2) F1	Fold 4 (Th1, Th2) F1	Fold 5 (Th1, Th2) F1	Mean. (Th1, Th2) F1
0.1	0.1, 0.1 0.53	0.2, 0.1 0.50	0.2, 0.1 0.49	0.2, 0.1 0.33	0.1, 0.1 0.57	0.16, 0.1 0.48
0.2	0.1, 0.1 0.53	0.2, 0.1 0.48	0.2, 0.1 0.44	0.2, 0.1 0.37	0.1, 0.1 0.57	0.16, 0.1 0.48
0.3	0.1, 0.1 0.53	0.2, 0.1 0.48	0.1, 0.1 0.48	0.1, 0.1 0.36	0.1, 0.1 0.59	0.12, 0.1 0.49
0.4	0.1, 0.1 0.54	0.1, 0.1 0.52	0.1, 0.1 0.47	0.1, 0.1 0.44	0.1, 0.1 0.60	0.1, 0.1 **0.52**
0.5	0.1, 0.1 0.55	0.2, 0.1 0.49	0.2, 0.1 0.46	0.2, 0.1 0.42	0.1, 0.1 0.59	0.16, 0.1 0.50
0.6	0.1, 0.1 0.56	0.1, 0.1 0.50	0.1, 0.1 0.47	0.1, 0.1 0.46	0.1, 0.1 0.57	0.1, 0.1 0.51
0.7	0.1, 0.1 0.57	0.1, 0.1 0.49	0.15, 0.1 0.43	0.1, 0.1 0.46	0.1, 0.1 0.55	0.11, 0.1 0.50
0.8	0.1, 0.1 0.55	0.1, 0.1 0.48	0.1, 0.1 0.47	0.1, 0.1 0.49	0.1, 0.1 0.55	0.11, 0.1 0.51
0.9	0.1, 0.1 0.55	0.1, 0.1 0.47	0.2, 0.1 0.45	0.1, 0.1 0.49	0.1, 0.1 0.55	0.12, 0.1 0.50
1.0	0.15, 0.1 0.57	0.2, 0.1 0.46	0.2, 0.1 0.45	0.15, 0.1 0.48	0.15, 0.1 0.55	0.17, 0.1 0.50

TABLE XVII: Complete hyperparameter sensitivity analysis using 5-fold cross-validation with DS1 Apps.

α	Fold 1 (Th1, Th2) F1	Fold 2 (Th1, Th2) F1	Fold 3 (Th1, Th2) F1	Fold 4 (Th1, Th2) F1	Fold 5 (Th1, Th2) F1	Mean. (Th1, Th2) F1
0.1	0.1, 0.3 0.9317	0.2, 0.3 0.9627	0.2, 0.3 0.9625	0.2, 0.3 0.9443	0.2, 0.3 0.9600	0.18, 0.3 0.9552
0.2	0.1, 0.4 0.9441	0.2, 0.35 0.9627	0.15, 0.35 0.9619	0.2, 0.35 0.9504	0.15, 0.35 0.9474	0.16, 0.36 0.9533
0.3	0.1, 0.4 0.9506	0.1, 0.4 0.9710	0.15, 0.4 0.9686	0.2, 0.35 0.9507	0.1, 0.4 0.9529	0.13, 0.39 0.9588
0.4	0.1, 0.4 0.9538	0.1, 0.4 0.9794	0.1, 0.4 0.9722	0.1, 0.4 0.9422	0.1, 0.4 0.9474	0.1, 0.4 0.9590
0.5	0.1, 0.35 0.9337	0.1, 0.4 0.9710	0.1, 0.4 0.9722	0.1, 0.4 0.9532	0.1, 0.4 0.9412	0.1, 0.39 0.9543
0.6	0.1, 0.4 0.9632	0.1, 0.4 0.9794	0.1, 0.4 0.9722	0.1, 0.4 0.9449	0.1, 0.4 0.9412	0.1, 0.4 0.9602
0.7	0.1, 0.4 0.9634	0.1, 0.4 0.9794	0.1, 0.4 0.9758	0.1, 0.4 0.9674	0.1, 0.4 0.9412	0.1, 0.4 0.9654
0.8	0.1, 0.4 0.9634	0.1, 0.4 0.9794	0.1, 0.4 0.9793	0.1, 0.4 0.9735	0.1, 0.4 0.9357	0.1, 0.4 **0.9663**
0.9	0.1, 0.4 0.9576	0.1, 0.4 0.9794	0.1, 0.4 0.9828	0.1, 0.45 0.9353	0.1, 0.4 0.9419	0.1, 0.41 0.9594
1.0	0.1, 0.4 0.9607	0.1, 0.4 0.9794	0.1, 0.4 0.9828	0.1, 0.4 0.9480	0.1, 0.4 0.9474	0.1, 0.4 0.9637

D. Obfuscation-Aware Weighting

We assign an obfuscation score $o_i^v \in [0, 1]$ to each block b_i^v to capture the degree of code transformation. This score is used to weight the contribution of each block during embedding aggregation.

1) Smali Blocks: For Smali code, the score is estimated from instruction-level patterns by considering control-flow operations, padding instructions, and semantic operations. The obfuscation score increases with control-flow and padding density, and decreases with semantic signals. We compute a raw score as $s_i^v = \alpha d_{\text{cf}} + \beta d_{\text{pad}} - \gamma d_{\text{sem}}$, where d_{cf}, d_{pad}, and d_{sem} denote the corresponding densities. The final score is obtained using the sigmoid function $o_i^v = \frac{1}{1+e^{-k s_i^v}}$.

2) Java Blocks: For Java code, the score is computed using syntactic patterns. Control-flow complexity is approximated using branching and looping constructs, while semantic activity is estimated using method calls. The raw score is computed as $s_i^v = \alpha d_{\text{cf}} - \gamma d_{\text{sem}}$ and normalized using the same sigmoid function, $o_i^v = \frac{1}{1+e^{-k s_i^v}}$.

E. Hyperparameter Tuning Results

This section presents the complete results of the 5-fold cross-validation used for hyperparameter selection discussed in Section III, complementing the summary reported in Table XVII. For each fold, we report the best-performing configuration in terms of the decay factor (α) and thresholds (τ_{class}, τ_{API}), along with the corresponding library-level F1-score. The results demonstrate consistent performance across folds, with $\alpha = 0.8$ and the selected thresholds frequently yielding the highest or near-highest F1-scores. Minor variations across folds reflect dataset-specific characteristics, but the overall trend confirms the robustness of the chosen configuration. These detailed results justify the selection of the final hyperparameters used in all subsequent experiments.

Table XVIII presents the full cross-validation results for DS2, for R8-obf-opt subsets. For each fold, we report the best-performing configuration in terms of $(\alpha, \tau_{\text{class}}, \tau_{\text{API}})$ along with the corresponding F1-score. Compared to DS1, the optimal parameters for DS2 exhibit a noticeable shift, particularly in the decay factor α and API threshold τ_{API}, reflecting the structural changes introduced by R8 optimization. In optimized apps, lower α and threshold values tend to perform better, indicating that aggressive code shrinking and obfuscation reduce the reliability of deeper structural signals. In contrast, non-optimized apps retain more stable patterns, resulting in slightly higher optimal thresholds. These results further support the need for dataset-specific tuning when handling heavily optimized Android applications.

F. CVEs of Top Android Vulnerable Third-Party Libraries

Table XIX presents the vulnerable TPLs used in the in-the-wild analysis (Section V-A1) and the related CVE information.

TABLE XIX: Top Android-relevant vulnerable third-party libraries.

TPL	CVEs
jsoup	CVE-2020-15250
okhttp	CVE-2021-0341
log4j	CVE-2023-26464, CVE-2022-23307, CVE-2022-23305
gson	CVE-2022-25647
guava	CVE-2023-2976
junit	CVE-2020-15250
httpclient	CVE-2020-13956
retrofit	CVE-2018-1000850, CVE-2018-1000844
jackson core	CVE-2025-52999
bouncycastle	CVE-2025-8916
commons-lang3	CVE-2025-48924
protobuf	CVE-2024-7254